11.1

3. Find a trivial lower-bound class for each of the following problems and indicate, if you can, whether this bound is tight.

a. finding the largest element in an array

b. checking completeness of a graph represented by its adjacency matrix

c. generating all the subsets of an n-element set

d. determining whether n given real numbers are all distinct

a) $a_1, a_2, a_3 \ldots, a_n$

We need to be processed all elements in the array to find largest elems.

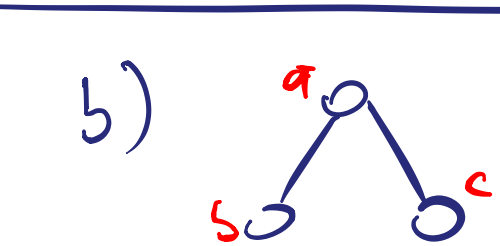
Trivial lower bound: $\Omega(n)$

Is this bound tight? Yes

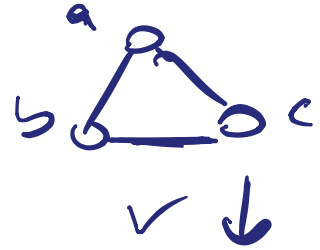
One pass scanning alg. $\rightarrow \Theta(n)$

---

b)

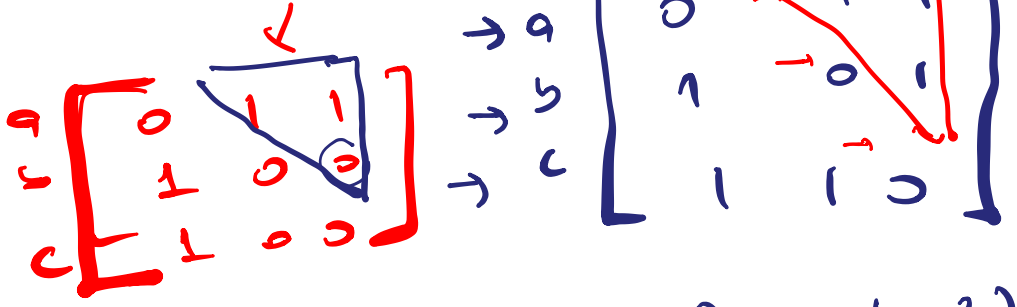


X



✓

→ check all elements in the upper triangle part of the matrix



$(n-1) + (n-2) + \cdots + (n-n-1) + 0$

$= (n-1) + (n-2) + \cdots + 1 = \frac{(n-1)\cdot n}{2}$

$\Omega(n^2)$ Tight

c) 2^n → Trivial lower bound

Tight ? Yes

---

d) $a_1, a_2, \ldots a_n$

Trivial lower bound → $\Omega(n)$ → To decide whether all elements are distinct, we need to process all elements in the array

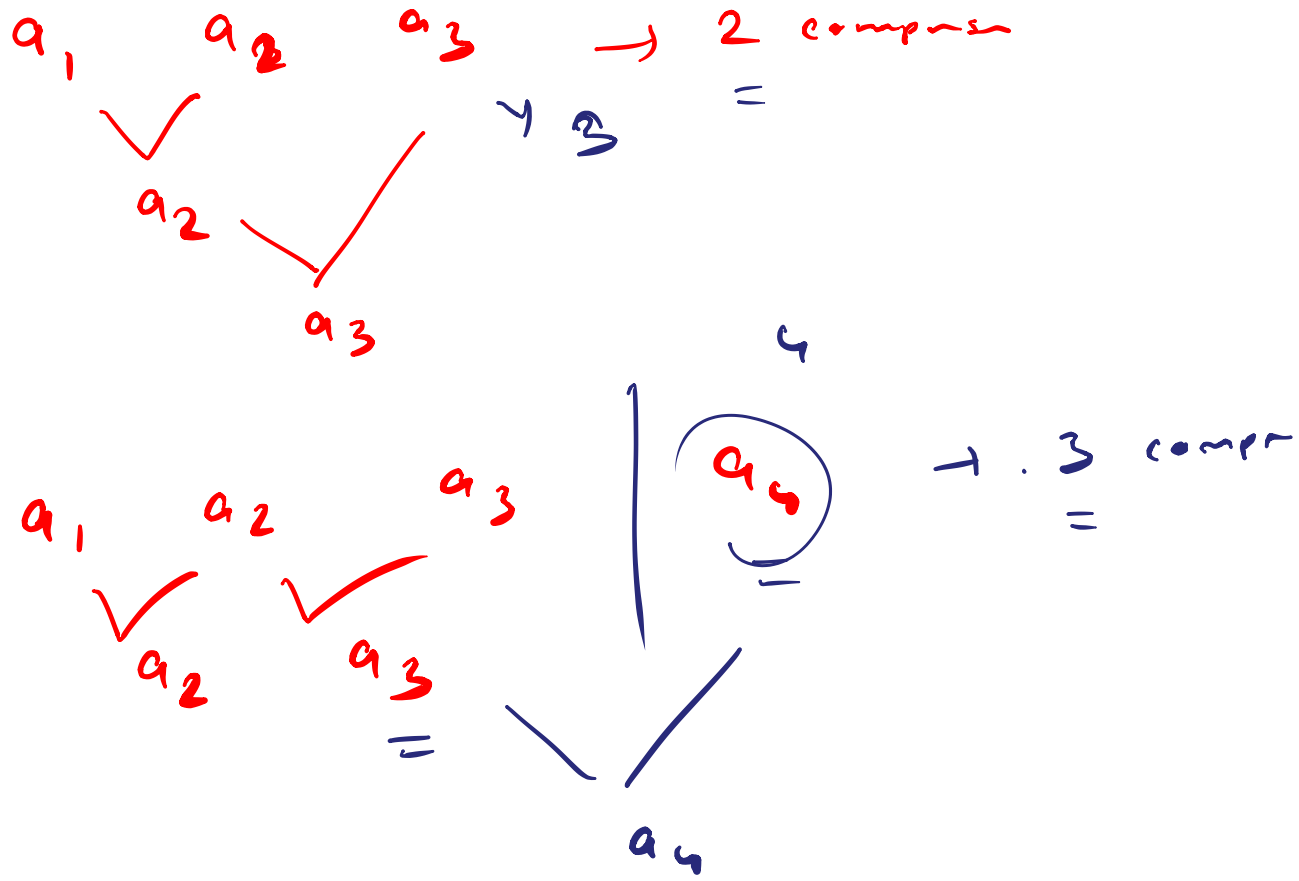
Presorting + compare → Known algorithm in $n \log n$

$n \log n$ $\quad$ n

So, it is not tight bound

11.3

5. Prove that any comparison-based algorithm for finding the largest element of an n-element set of real numbers must make $n - 1$ comparisons in the worst case.

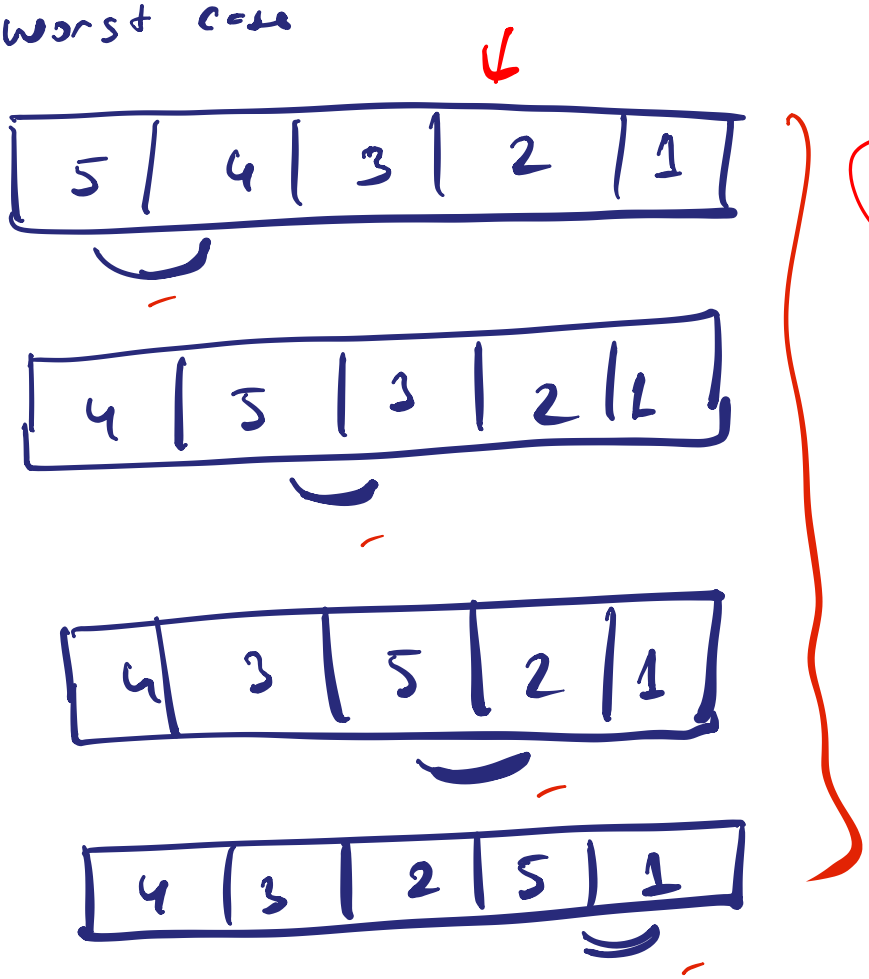


If less than $n-1$ comparisons are made at least two elements will be uncompared

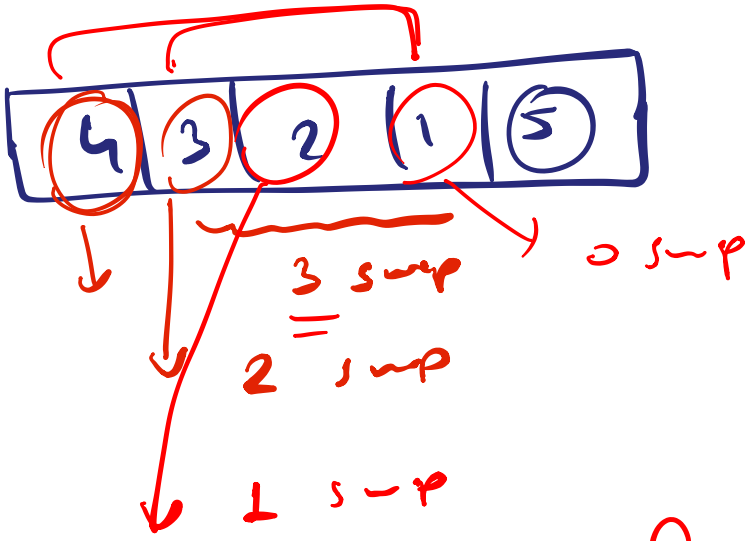
→ So, we must make $n-1$ comparisons in order to say which one is largest.

6. Find a tight lower bound for sorting an array by exchanging its adjacent elements.

worst case



Tight? Yes



$$\#\text{ swaps} = \sum_{i=1}^{n} (i-1) = 0+1+2+\dots+(n-1)$$

$$= \frac{(n-1)\cdot n}{2}$$

$$\Omega(n^2) \quad //$$

11.1

11. Find a tight lower-bound class for the problem of finding two closest numbers among n real numbers $x_1, x_2, \ldots, x_n$.

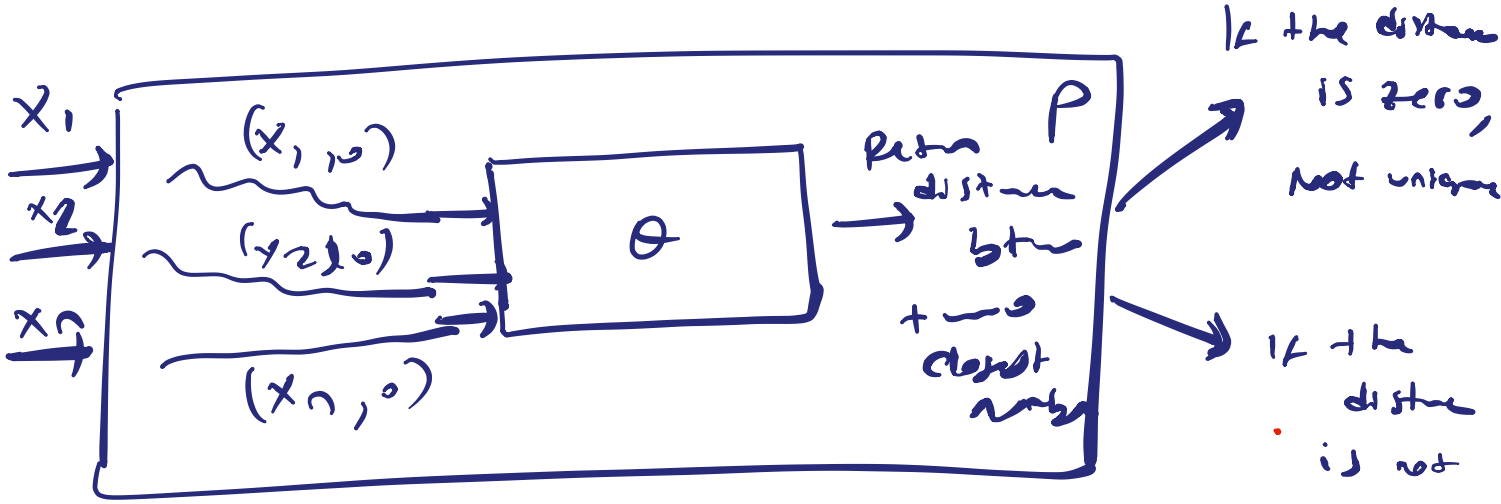
Θ

$x_1, x_2, \ldots, x_n \rightarrow (x_1, 0), (x_2, 0) \ldots (x_n, 0)$

→ Element Uniqueness (P) → $n \lg n$ (!)

↓ Reduce

closest numbers (Θ)



Lower bound for element uniqueness is also $\Omega(n \lg n)$ → lower bound for closest pair problem

Is this tight? Yes

↳ presorting based alg.

9.1.2

2. Consider the problem of finding the median of a three-element set $\{a, b, c\}$ of orderable items.

a. What is the information-theoretic lower bound for comparison-based algorithms solving this problem?

b. Draw a decision tree for an algorithm solving this problem.

c. If the worst-case number of comparisons in your algorithm is greater than the information-theoretic lower bound, do you think an algorithm matching the lower bound exists? (Either find such an algorithm or prove its impossibility.)

a) 3 elements

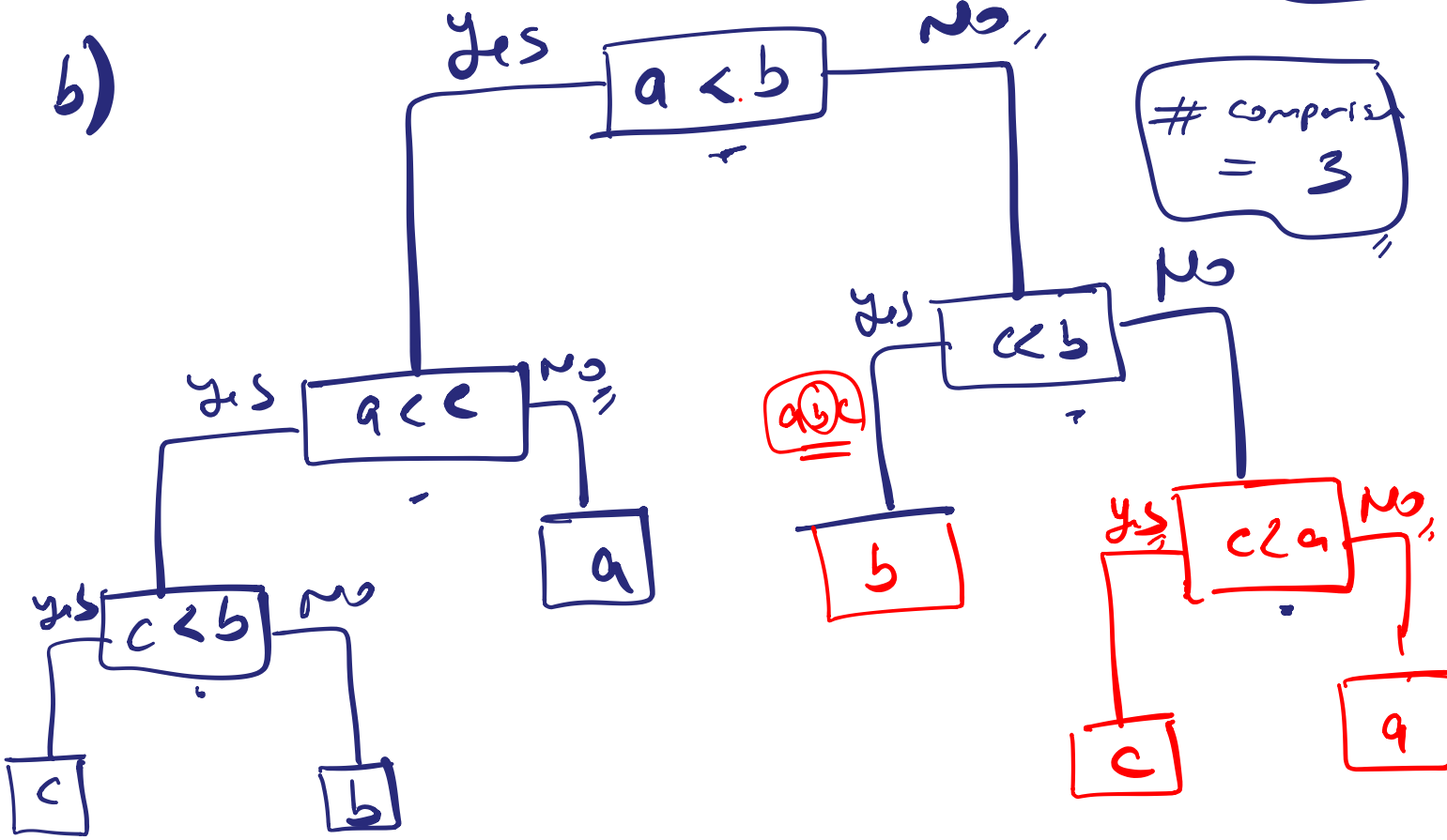
possible outcomes : a, b, c

outcomes = 3

information theoretic lower bound

$$W_n(n) \geq \lceil \log_2 \#\text{outcomes} \rceil = \lceil \log_2 3 \rceil = 2$$

b)



c) # comparisons in worst case > Information theoretic bound

→ It is impossible to find the median of three real numbers in less than three comparisons.

→ Suppose

- Compare a and b; (one winner)
- Compare winner and c

If $a < b$

$b < c$ the median is b ✓

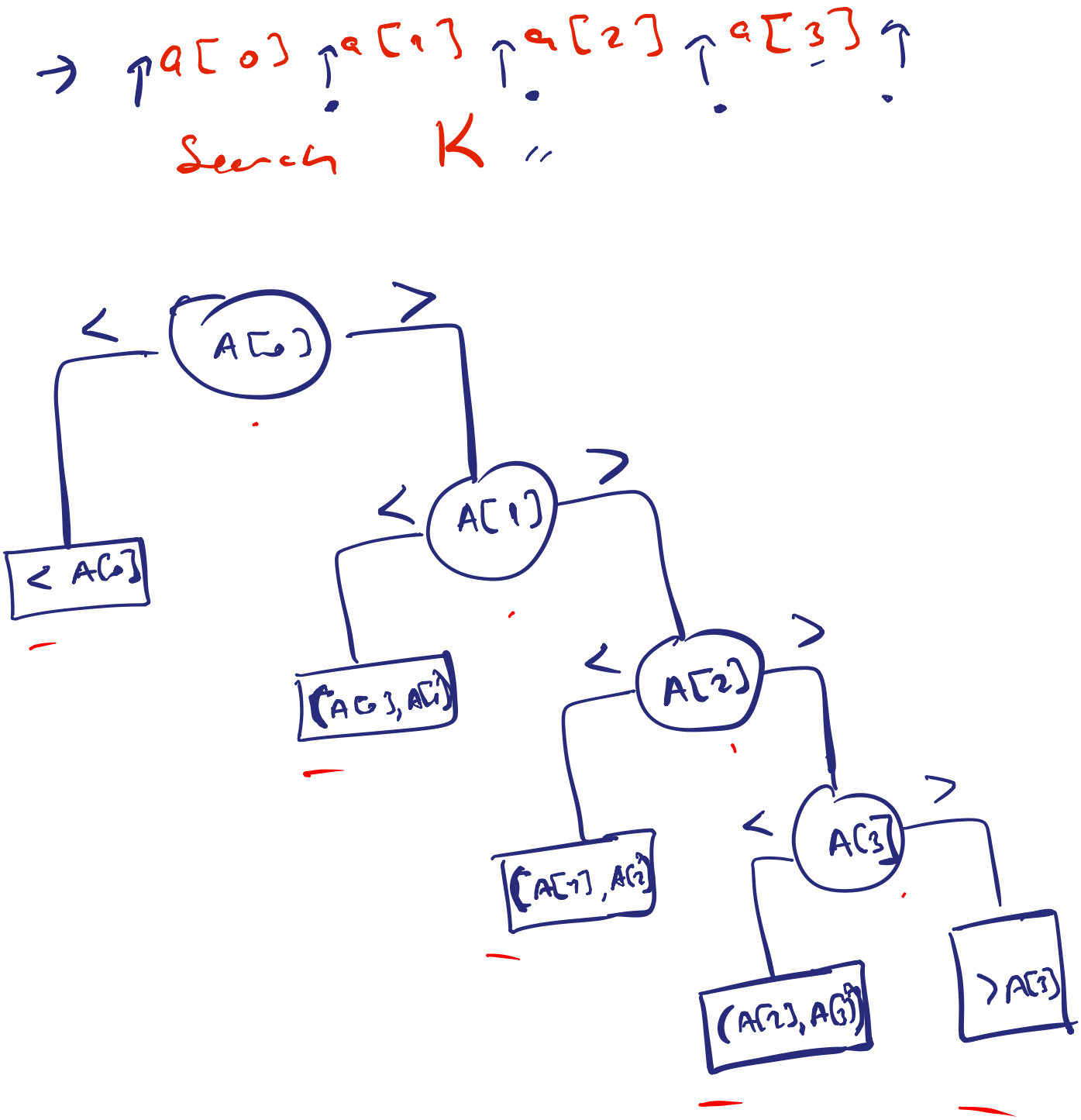
But if $a < b$

$c < b$ → we do not know whether median is a or c

→ we need to compare a and c

11.2

6. Draw a binary decision tree for searching a four-element sorted list by sequential search.



11.2

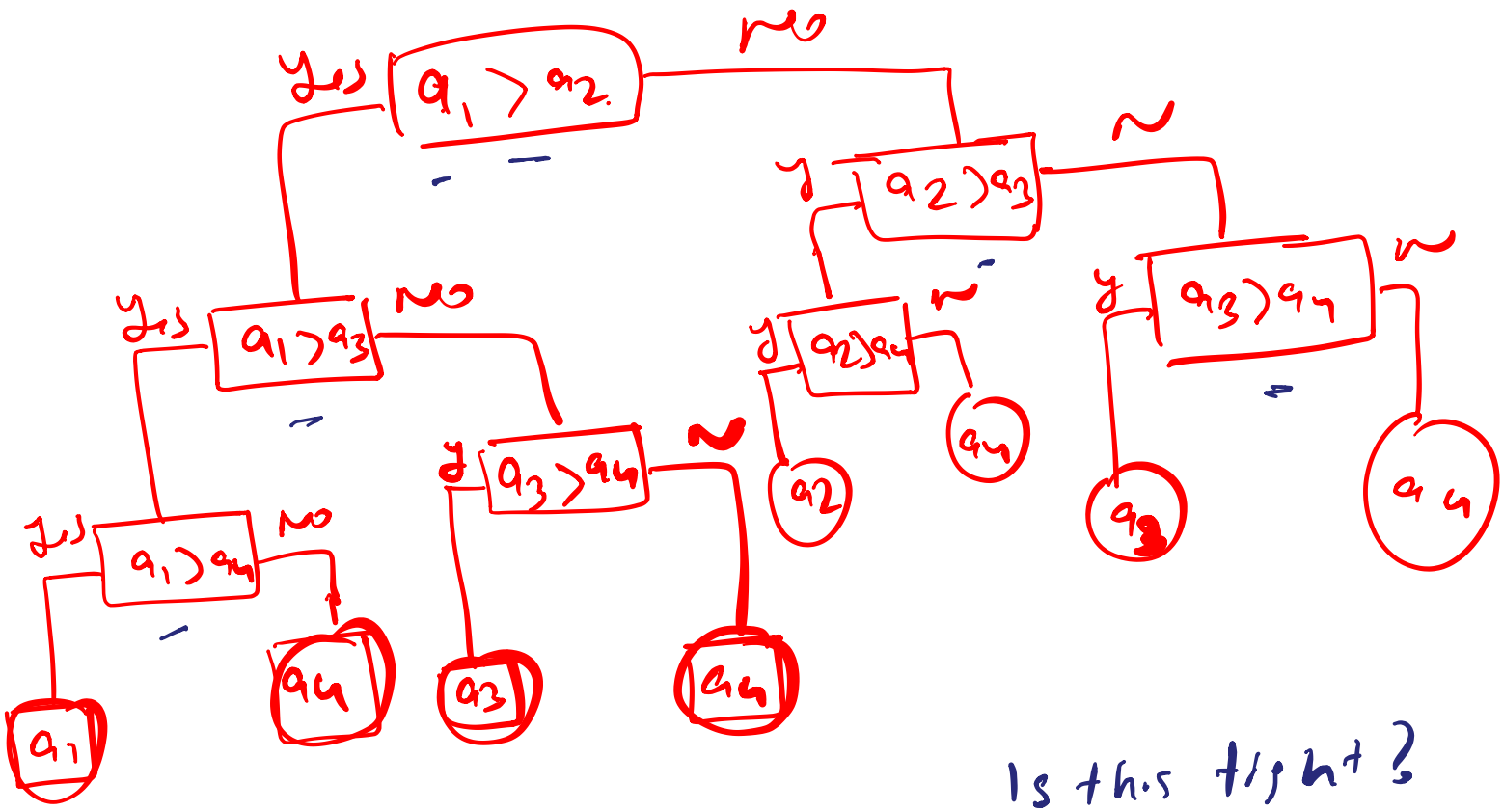
8. What is the information-theoretic lower bound for finding the maximum of n numbers by comparison-based algorithms? Is this bound tight?

$a_1, a_2, \ldots, a_n$

outcomes: $n!$

Theoretic lower bound $= \lceil \log_2 n! \rceil$ ← # outcomes

$a_1, a_2, a_3, a_4 \rightarrow \lceil \log_2 4! \rceil = 5$



Comparison in worst case $= 3$

Is this tight?

No

At least $n-1$ comparisons are needed.

11.3

2. A certain problem can be solved by an algorithm whose running time is in $O(n^{\log_2 n})$. Which of the following assertions is true?

a. The problem is tractable.

b. The problem is intractable.

c. Impossible to tell.

a) $n^{\log_2 n}$ grows infinitely faster than any polinomial function n^k.

$O(n^{\log_2 n})$ → upper bound

→ There may be an algoritm that can solve the problem in polinomial time.

So, → UNKNOWN

b) UNKNOWN

c) Yes

Suppose that Problem X can be reduced to Shortest Path problem and Problem Y can be reduced to Traveling Salesman problem. All reductions can be done in linear (O(n)) time. For each of the following, indicate whether it is True, False, or Unknown. Give your reasoning, otherwise your answer will not be graded.

(i) Problem Y is more difficult than Prob. X.

(ii) Traveling Salesman problem can be reduced to Problem X in linear time.

(iii) Both Problem X and Problem Y can be tractable problems.

i) Problem X → SPP $O(n^2)$ → in P

Problem Y → TSP → NP

UNKNOWN

ii) TSP → Problem X (in P)

→ If TSP (NP) can be reduced in problem X in polinomal, problem X would be in NP

FALSE, UNKNOWN//

iii) Problem X → TRACTABLE

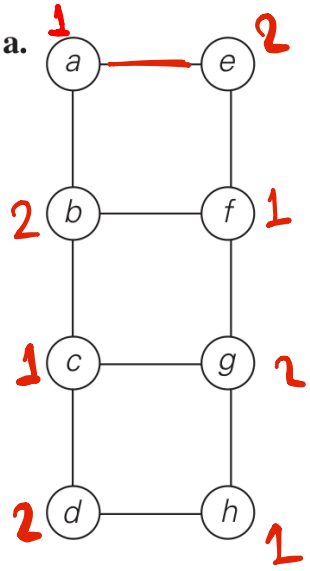
(P)

Problem Y → UNKNOWN

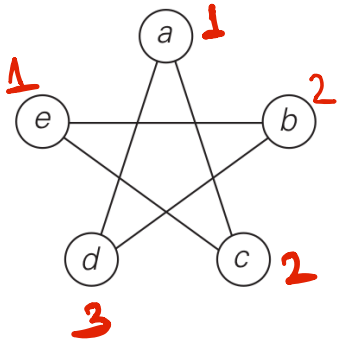
11.3

4. For each of the following graphs, find its chromatic number.

a.

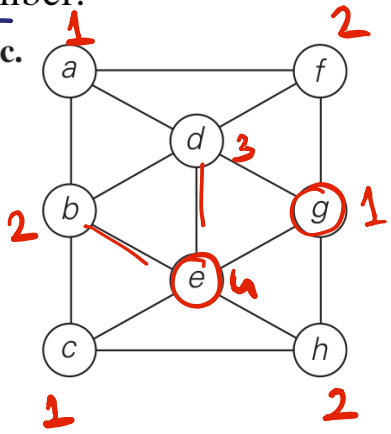


b.



chromatic number = 3

c.

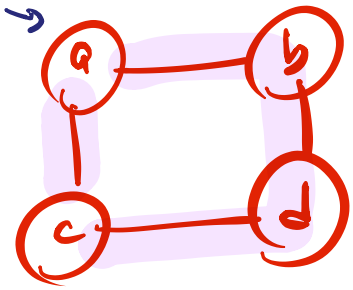


chromatic number = 4

4 Chromatic number = 2

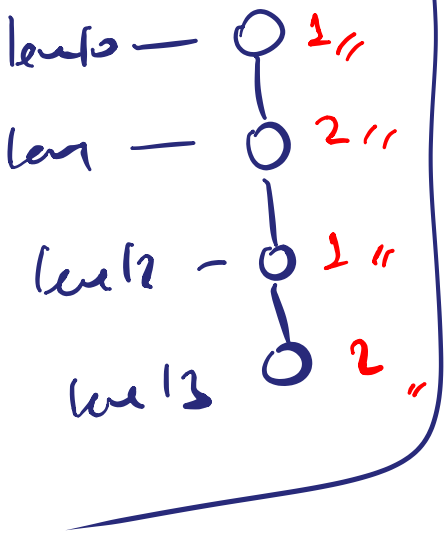
11.3

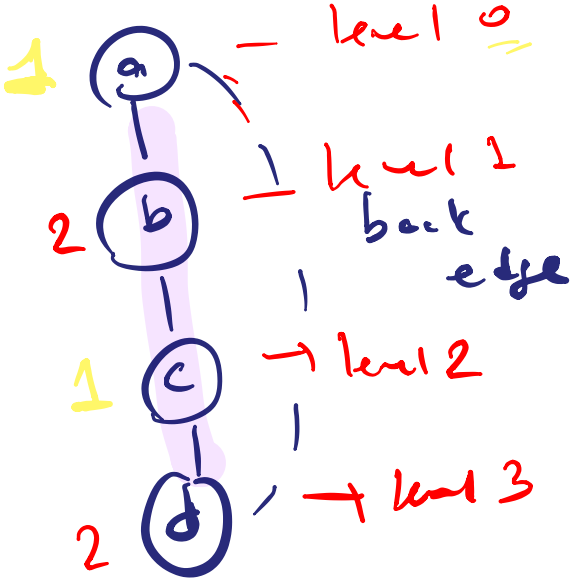
5. Design a polynomial-time algorithm for the graph 2-coloring problem: determine whether vertices of a given graph can be colored in no more than two colors so that no two adjacent vertices are colored the same color.



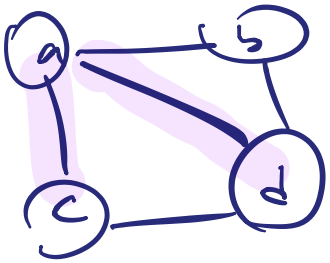
Create DFS Forest

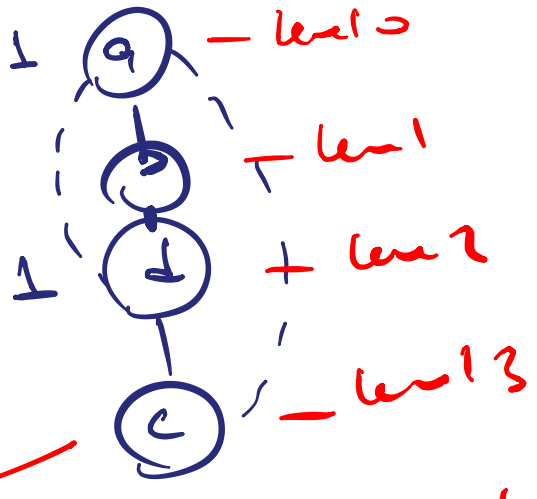
d
c
b
a





DFS Forest



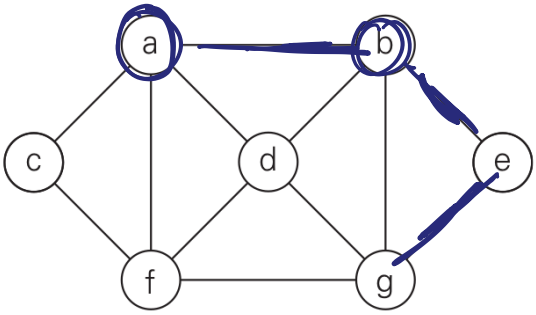


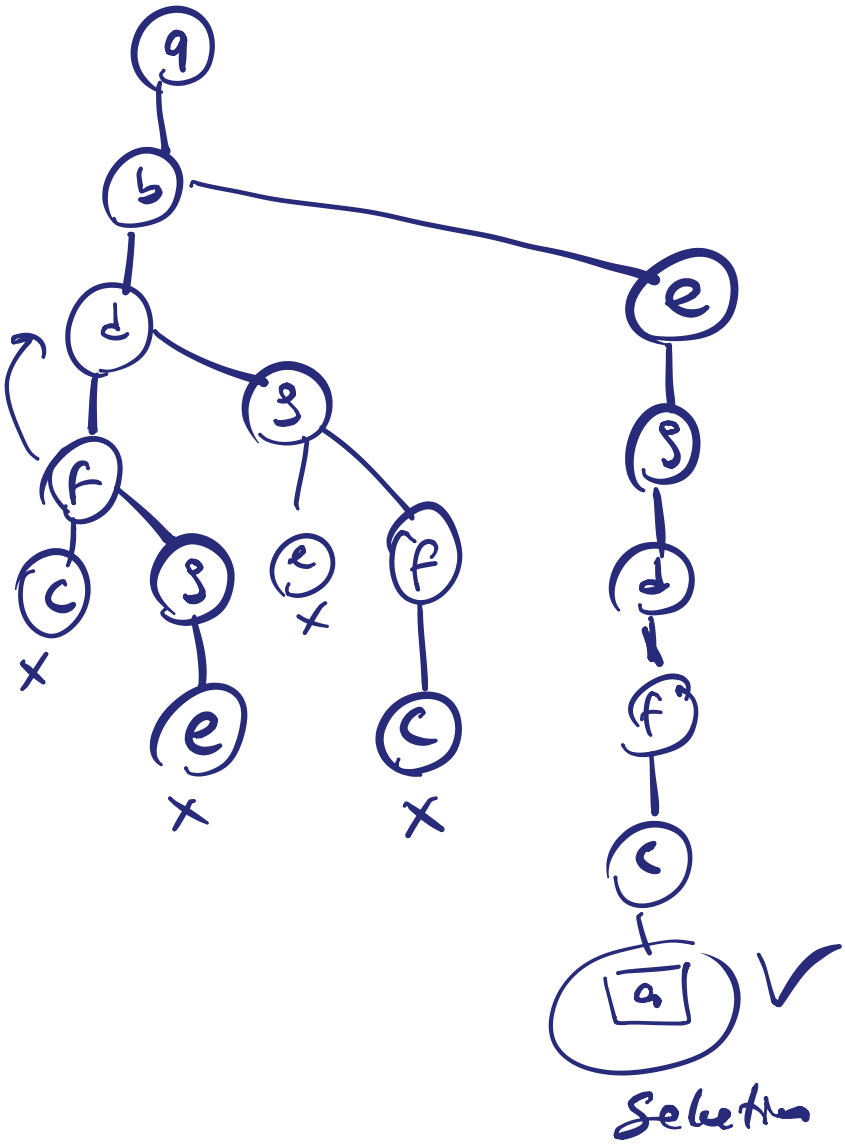
The nodes in the even levels are colored in the same color. So it is not 2-colorable.

Our Algorithm.

- Create a DFS forest.
- If DFS forest has no back edges, the graph's vertices can be colored in two colors by alternating the colors on odd and even levels of the forest.
- If the DFS forest has back edges, it is 2-colorable iff every back edges connects vertices on the levels of different parity; one on an odd level and the other on an even level.

5. Apply backtracking to the problem of finding a Hamiltonian circuit in the following graph.





6. Apply backtracking to solve the 3-coloring problem for the graph in Figure 12.3a.

